



MODULE MIA4 • JOUR 3 / 5

Réplication & Sharding

Jour 3 — Haute disponibilité et scalabilité horizontale

Replica Sets • Élection du primary • Read/Write Concerns • Sharding • Shard key • Cluster complet

Plan de la journée

7 heures effectives · théorie + démonstration d'un cluster complet en Docker

- 1 **Rappel Jour 2** — 9h00-9h30
- 2 **Pourquoi scaler ?** vertical vs horizontal · 9h30-10h30
- 3 **Replica Set : principes & élection** · 10h45-12h00
-  *Pause déjeuner — 12h00-14h00*
- 4 **Read / Write Concerns** — 14h00-14h45
- 5 **Sharding : concept & shard key** · 14h45-15h45
- 6 **Cluster : Replica + Sharding** · 15h45-16h30
- 7 **Atelier — Monter un Replica Set** · 16h30-17h00

HORAIRES

9h00 → 17h00

 7h effectives

 Pause 10h30-10h45

 Déjeuner 12h-14h

 1 démo cluster

CE QUE VOUS SAUREZ FAIRE

- ✓ Monter un Replica Set 3 nœuds
- ✓ Provoquer un failover
- ✓ Choisir une shard key
- ✓ Lire une topo cluster
- ✓ Ajuster Read/Write Concern

Rappel Jour 2 & ouverture Jour 3

🕒 Ce que nous avons couvert

- ✓ **Schema design** : piloté par les requêtes de l'appli.
- ✓ **Embed vs Reference** : cardinalité, taille bornée, cycle de vie.
- ✓ **6 patterns** : Bucket, Subset, Computed, Polymorphic, Versioning, Extended Ref.
- ✓ **Indexation** : règle **ESR** · index simple, composé, multi-clés, text, géo, TTL.
- ✓ **explain()** : IXSCAN >> COLLSCAN · ratio docsExamined/nReturned.
- ✓ **Transactions** multi-doc depuis 4.0 · à utiliser avec parcimonie.
- ✓ **PyMongo** : un seul MongoClient par app, API identique à mongosh.

📈 Ouverture — pourquoi Jour 3 ?

Nos schémas sont bons, nos index efficaces — mais tout tient sur **un seul serveur**.

2 questions critiques

1. Que se passe-t-il si **ce serveur tombe** ?
2. Que se passe-t-il quand la BDD **dépasse la capacité** d'un seul serveur ?

- » **Réplication** = tolérance aux pannes
- » **Sharding** = scalabilité horizontale
- » Les deux se **combinent** en production.

Scaler : vertical vs horizontal

Deux stratégies pour absorber plus de charge. Les SGBDR historiques n'en connaissent qu'une.

↑ Scale UP — vertical

Grossir la machine : + CPU, + RAM, + SSD, + IOPS.

Avantages : simple, pas de changement d'archi.

Limites :

- Coût **non-linéaire** (une machine "haut de gamme" coûte 5× plus qu'une standard mais ne fait que 2×)
- Plafond matériel (max ~128 vCPU, 4 To RAM)
- **SPOF** — un seul serveur = une seule panne pour couler tout

↔ Scale OUT — horizontal

Ajouter des machines : 3, 10, 100 serveurs modestes en cluster.

Avantages :

- Coût **linéaire** (matériel standard)
- Pas de plafond théorique
- Pannes tolérées (redondance native)

Difficultés : complexité opérationnelle, cohérence distribuée, réseau.

📊 Les 2 dimensions de la charge

Volume de données — la BDD ne tient plus sur un disque

→ solution : **Sharding** (répartir les données)

Volume de requêtes — trop de lectures/écritures simultanées

→ solution : **Réplication** (répartir les lectures)

Disponibilité — la BDD ne doit jamais tomber

→ solution : **Réplication** (auto-failover)

Résilience géographique — panne de datacenter

→ solution : **Réplication multi-région**

Réplication MongoDB — les principes

Répliquer = maintenir plusieurs copies synchronisées d'une même donnée sur des serveurs distincts.

📖 Vocabulaire de base

Replica Set — un ensemble de nœuds MongoDB (typiquement 3) hébergeant les mêmes données.

Primary — le seul nœud qui accepte les écritures. Une élection le désigne.

Secondary — nœud qui réplique le primary de manière asynchrone via l'**oplog**.

Oplog — collection **capped** (taille fixe, circulaire) qui contient chaque opération d'écriture du primary.

Arbiter — nœud sans donnée dont le seul rôle est de voter lors des élections (nombre impair).



Haute disponibilité

Si le primary tombe, un secondary devient automatiquement primary en 10-30s. **Zéro intervention humaine.**



Distribution des lectures

Les lectures peuvent aller sur les secondaries via `readPreference` — soulage le primary.



Résilience géographique

Répartir les nœuds entre régions/zones cloud pour tolérer une panne de datacenter complet.

Topologie d'un Replica Set 3 nœuds



Flèches implicites

Le primary **pousse** l'oplog vers les secondaries.
Les secondaries envoient des **heartbeats** toutes les 2s pour surveiller le cluster.

```
# 1) Démarrer 3 mongod en mode replica set  
# --replSet rs0 sur les 3 nœuds
```

```
mongod --replSet rs0 --port 27017 \  
      --dbpath /data/db1 --bind_ip_all
```

```
# 2) Se connecter et initialiser le RS  
mongosh --port 27017  
> rs.initiate({  
  _id: "rs0",  
  members: [  
    { _id:0, host:"mongo-1:27017" },  
    { _id:1, host:"mongo-2:27017" },  
    { _id:2, host:"mongo-3:27017" }  
  ]  
})
```

```
# 3) Diagnostic  
> rs.status()      # état global  
> rs.conf()        # configuration  
> rs.isMaster()    # qui est primary ?
```

```
# 4) Ajouter un secondary à chaud  
> rs.add("mongo-4:27017")
```

Élection du Primary — protocole Raft

Quand le primary devient injoignable, les secondaries se coordonnent pour en élire un nouveau. MongoDB implémente une variante de **Raft**.

Étapes de l'élection (typiquement 10-30 s)

1

Détection

Un secondary ne reçoit plus de heartbeat du primary pendant **electionTimeoutMillis** (10 s par défaut).

2

Candidature

Le secondary devient **candidate**, incrémente son **term**, demande des votes aux autres membres.

3

Vote majoritaire

Il faut la **majorité stricte** des membres pour être élu (2/3, 3/5, 4/7). D'où le nombre **impair**.

4

Nouveau primary

Le candidat élu devient primary et se met à accepter les écritures. Les drivers redirigent automatiquement.

💡 Critères pour être élu

- Priorité > 0 (configurable par membre)
- **Oplog le plus à jour** — le candidat avec les données les plus récentes
- Membre **votant** (au moins 1 vote configuré)
- Peut communiquer avec la **majorité** du set

⚠ Split-brain — pourquoi impair

Avec 4 nœuds séparés par une partition réseau en 2 groupes de 2 : **aucun n'a la majorité** → aucune écriture possible → panne complète.

Avec 3 nœuds : le groupe de 2 conserve le quorum, le nœud isolé passe en read-only.

Règle : 3, 5 ou 7 membres — jamais un nombre pair sans arbitre.

Write Concern & Read Concern & Read Preference

Trois curseurs qui ajustent finement le compromis **cohérence** ↔ **latence** ↔ **disponibilité**.

Write Concern {w: ...}

Combien de **nœuds doivent confirmer** l'écriture avant retour.

```
{ w: 1 }
```

Seul le primary ack. Rapide, risque de perte si failover immédiat.

```
{ w: "majority" }
```

La **majorité** des votants ack. **Défaut recommandé**. Aucune perte tolérée.

```
{ w:"majority", j:true }
```

Ack seulement quand écrit sur disque (**journal**) — encore plus sûr.

Read Concern

Fraîcheur attendue des données lues.

```
"local"
```

Défaut. Retourne les données locales, même non-confirmées par la majorité.

```
"majority"
```

Ne renvoie que ce qui a été confirmé par la **majorité** — pas de rollback possible.

```
"linearizable"
```

La plus stricte. Garantit la lecture après toute écriture confirmée. **Coûteuse**.

Read Preference

Sur **quel type de nœud** lire ?

```
primary
```

Défaut. Cohérence forte.

```
primaryPreferred
```

Primary sauf s'il est indispo.

```
secondary
```

Décharge le primary — data éventuellement **stale**.

```
nearest
```

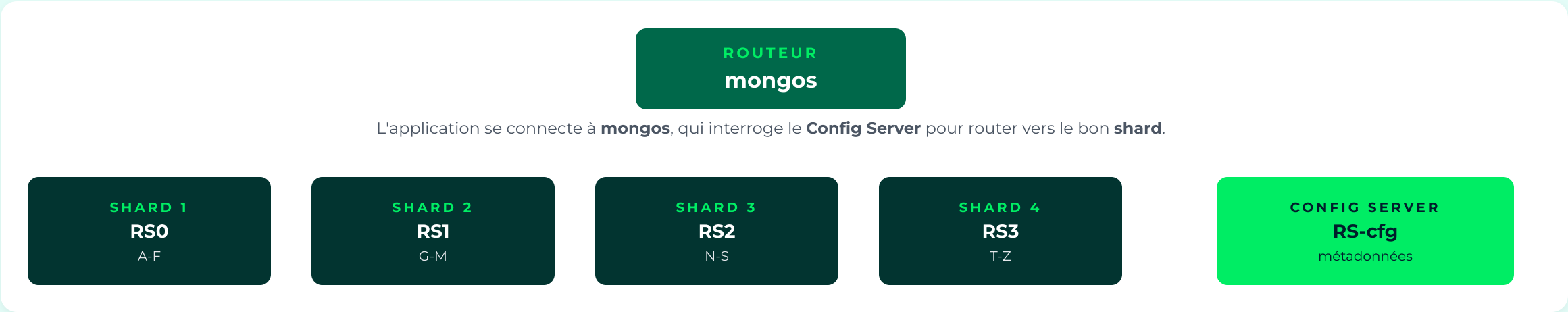
Le nœud avec la **plus faible latence** réseau.

Sharding — répartir les données sur N shards

Chaque **shard** contient un **sous-ensemble** des documents d'une collection. MongoDB route automatiquement les requêtes.

📖 Vocabulaire du sharding

- Shard** — un sous-cluster (typiquement un Replica Set) qui stocke une partie des données.
- mongos** — routeur, point d'entrée. Il n'a pas de données, il redirige les requêtes vers le bon shard.
- Config Server** — Replica Set spécial qui stocke les **métadonnées** du cluster (quel document est sur quel shard).
- Chunk** — plage contiguë de shard keys — l'unité de découpage et de migration entre shards.
- Balancer** — processus interne qui migre les chunks entre shards pour équilibrer la charge.



Shard Key — la décision la plus importante

Le champ (ou combinaison) qui détermine sur **quel shard** chaque document est stocké. Le mauvais choix compromet tout le cluster.

👍 Critères d'une bonne shard key

Cardinalité élevée — beaucoup de valeurs distinctes (pas de champ booléen !)

Distribution uniforme — pas de valeur "chaude" concentrant 80% du trafic

Écriture répartie — éviter les monotoniques croissantes (timestamps)

Alignée avec les requêtes — les `find()` fréquents doivent inclure la shard key

Immuable (avant 4.2) — modifiable depuis 4.2 mais coûteux

👎 Anti-patterns fréquents

timestamp / _id croissant — un seul shard reçoit toutes les écritures (**hot shard**)

booléen / champ à faible cardinalité — pas assez de valeurs pour répartir

champ qui n'apparaît jamais en requête — chaque find déclenche un

scatter-gather sur tous les shards

shard key changeante — force la migration du document entre shards

🔗 Deux stratégies de sharding

Ranged (plages)

Découpage par plages contiguës de valeurs. Idéal pour requêtes de type `$gte/$lt`. Risque de hot shard si les données arrivent triées.

```
sh.shardCollection("db.coll", { user_id: 1 })
```

Hashed (hachée)

MongoDB applique un hash MD5 sur la shard key. Distribution **quasi-uniforme** — mais requêtes par plage impossibles.

```
sh.shardCollection("db.coll", { user_id: "hashed" })
```

Cheminement d'une requête dans un cluster



Étapes : ① App → mongos ② mongos consulte Config Server pour trouver le shard cible ③ mongos route vers le bon shard ④ Le shard exécute la requête ⑤ Résultats agrégés retournés à l'app

Pour l'application, le cluster ressemble à un serveur MongoDB unique — la complexité est cachée.

🔍 Targeted vs Broadcast query

Targeted — la requête inclut la shard key → mongos route vers 1 seul shard.
Optimal.

Broadcast (scatter-gather) — la shard key est absente → mongos interroge **tous** les shards et agrège. Coûteux.

Objectif : ≥ 90% des requêtes fréquentes en targeted.

🛠 Dimensionnement minimum production

- **Config Server** : Replica Set 3 nœuds (obligatoire)
- **mongos** : 2+ instances (redondance, load balancing app-side)
- **Chaque shard** : Replica Set 3 nœuds
- **Minimum** : $3 + 2 + 3 \times N = 14$ machines pour un cluster 3 shards

Coupler Réplication et Sharding — la mise en œuvre

En production, un cluster combine les deux : chaque shard est un Replica Set — meilleur des deux mondes.

```
# 1) Config Server Replica Set
mongod --configsvr --replSet cfgRS \
  --port 27019 --dbpath /data/cfg1
# ... idem pour cfg2, cfg3
mongosh --port 27019
> rs.initiate({_id:"cfgRS",configsvr:true,
  members:[{_id:0,host:"cfg1:27019"},...]}))

# 2) Shard 1 Replica Set
mongod --shardsvr --replSet shard1RS \
  --port 27018 --dbpath /data/s1a
# ... idem s1b, s1c
mongosh --port 27018
> rs.initiate({_id:"shard1RS",members:[...]})

# 3) mongos — routeur
mongos --configdb "cfgRS/cfg1:27019,cfg2:27019,cfg3:27019" \
  --port 27017

# 4) Enregistrer le shard dans le cluster
mongosh --port 27017
> sh.addShard("shard1RS/s1a:27018,s1b:27018,s1c:27018")

# 5) Activer le sharding sur une DB puis une collection
> sh.enableSharding("ecommerce")
> sh.shardCollection(
  "ecommerce.commandes",
  { client_id: "hashed" }
)

# 6) Vérifier
> sh.status()
```

✓ Bénéfices du couplage

Haute disponibilité · si un nœud du shard tombe, un secondary prend le relais

Scalabilité · ajout de shards pour absorber croissance des données

Distribution des lectures · secondaries de chaque shard servent les lectures

Résilience · perte tolérée jusqu'à minorité de chaque shard

⚠ Coûts & complexité

- **14+ machines** minimum pour un cluster 3 shards
- **Opérations multi-shards** plus complexes (aggregations, transactions)
- **Monitoring** avec Ops Manager / Cloud Manager / Prometheus
- **Backup** cohérent nécessite outillage spécifique (**mongodump --oplog**)
- À éviter tant que **Replica Set seul** suffit

Suivi des performances liées au Sharding

```
// État global du cluster
sh.status()

// Nombre de chunks par shard
db.chunks.aggregate([
  { $group: { _id: "$shard",
              n: { $sum: 1 } } }
])

// Distribution d'une collection
db.commandes.getShardDistribution()

// Balancer activé ?
sh.getBalancerState()
sh.isBalancerRunning()

// Historique des migrations
use config
db.changeLog.find({
  what: "moveChunk.commit"
}).sort({ time: -1 }).limit(10)

// Analyser une requête sur cluster
db.commandes.find({client_id:"C42"})
  .explain("executionStats")
// Regarder "shards" dans la sortie
// SingleShard vs SHARD_MERGE (scatter)
```

Un cluster mal équilibré est aussi mauvais que pas de cluster. Voici les métriques à suivre.

📈 Métriques clés à surveiller

Nombre de chunks par shard — écart max/min < 8

Ratio jumbo chunks — chunks trop gros à split (> 128 Mo par défaut)

Balancer round duration — le rééquilibrage doit rester rapide

Targeted vs Broadcast ratio — objectif > 90% targeted

Chunks migrations / heure — trop de migrations = shard key mal choisie

⚠️ Signes qu'il faut agir

Un shard concentre > 60% des données → **revoir shard key**

Migrations en boucle qui n'aboutissent pas → **désactiver balancer** temporairement

Requêtes lentes malgré index → vérifier **SingleShard vs Broadcast**

Espaces disque déséquilibrés → contrôler **chunks migrations**

Alertes OpLog près saturation → **augmenter oplogSize**

ATELIER GUIDÉ 1H

TP — Monter un Replica Set + provoquer un failover

📋 Consignes

- 1 **docker-compose** — écrire un fichier avec 3 services `mongo1`, `mongo2`, `mongo3` tous en `--replSet rs0`.
- 2 **Initialiser** le Replica Set avec `rs.initiate(...)`. Attendre que `rs.status()` montre 1 PRIMARY + 2 SECONDARY.
- 3 **Insérer 1 000 documents** dans la collection `events` avec `w:"majority"`.
- 4 **Vérifier la réplication** — se connecter à un secondary, faire `rs.secondary0k()` puis lire.
- 5 **Failover manuel** — `docker stop mongo1` (le primary). Mesurer le temps avant qu'un secondary ne devienne primary.
- 6 **Redémarrer** `mongo1`. Il doit rejoindre en tant que **secondary** et se resynchroniser via oplog.
- 7 **Bonus** : écrire un script PyMongo qui écrit en boucle et affiche à quel moment il tombe en `NotPrimaryError`, puis se reconnecte automatiquement.

🚩 LIVRABLES

- `docker-compose.yml`
- `init-rs.js` — initialisation
- `bench-failover.md` — mesures
- Capture `rs.status()` avant/après
- Script `retry-writer.py` (bonus)

💡 ÉVALUATION

- docker-compose fonctionnel
- Compréhension du **quorum**
- Analyse du temps de failover
- Explication d'un split-brain potentiel
- Bonus : gestion propre de **NotPrimaryError**

Ce qu'il faut retenir du Jour 3



Scale OUT, pas UP

MongoDB est conçu pour grossir horizontalement — bien plus économique passé un certain volume.



Replica Set : 3 nœuds impairs

Nombre impair OBLIGATOIRE pour le quorum. Failover automatique en 10-30 s via Raft.



w:"majority" par défaut

Toujours privilégier majority pour les écritures critiques. Pas de perte de donnée tolérée.



Shard key = décision critique

Cardinalité, distribution, présente dans les requêtes. Un mauvais choix ruine le cluster.



Targeted >> Broadcast

Une requête sans shard key = scatter-gather sur tous les shards. Objectif : ≥ 90% targeted.



Sharding = production seulement

14+ machines mini. Complexe. À éviter tant que Replica Set seul suffit.

→ **Demain — Jour 4** : Aggregation Pipeline, GridFS, index géospatiaux, diagnostic MongoDB, backup et QCM final.